

Práctica Final Integradora - Grafos

Estructuras de Datos y Algoritmos I

Cristóbal Machado Sánchez

Universidad EAFIT - 2026

Introducción.

En esta práctica se trabajó con el dataset roadNet-PA que representa la red de carreteras del estado de Pennsylvania en Estados Unidos. Dicho dataset está disponible en Stanford SNAP y tiene alrededor de 1 millón de nodos, que representan las intersecciones, y 1.5 millones de aristas, representando las carreteras.

El objetivo es implementar algoritmos de grafos vistos en clase sobre este dataset real, especialmente análisis estructural, búsqueda de caminos más cortos y construcción de árboles generadores mínimos.

Lo más interesante del trabajo fue ver cómo los algoritmos, cómo el de Dijkstra que en clase se ven con grafos pequeños de 5 o 10 nodos, funcionan igual sobre un millón de nodos solo que más lento por obvias razones.

Dataset y representación.

El grafo se representó usando una lista de adyacencia y no matriz de adyacencia porque con 1 millón de nodos una matriz necesitaría almacenar $1,088,092 \times 1,088,092$ valores, lo que serían aproximadamente 4.7 terabytes de memoria. Claramente eso no es viable, entonces la lista de adyacencia es la única opción práctica para grafos de este tamaño.

Como el dataset original no tiene pesos en las aristas, se asignaron pesos aleatorios entre 1 y 10 usando `srand(42)`, esto para que los resultados sean reproducibles.

Los IDs de los nodos en el dataset no son consecutivos, van saltando, por lo que había que reindexarlos internamente de 0 a $n-1$ para poder usar vectores normales. Luego se guardó un mapa entre el ID original y el índice interno para poder reportar los resultados correctamente.

Resultados Módulo A.

Al cargar y analizar el grafo se obtuvieron los siguientes resultados:

- Nodos: 1,088,092
- Aristas: 1,541,898
- Componentes conexas: 206
- Nodos en la componente principal: 1,087,562
- Grado promedio: 2.834
- Diámetro aproximado: 592

Comparando con los valores publicados por SNAP, los nodos y aristas coinciden exactamente, al igual que la componente conexa principal. El grado promedio de 2.834 tiene sentido para una red vial porque la mayoría de intersecciones conectan 2 o 3 carreteras.

El diámetro que reporta SNAP es 782 pero el obtenido fue 592. Esto pasa porque calcular el diámetro exacto requeriría hacer BFS desde todos los nodos, lo cual tardaría demasiado, por lo que se se hizo BFS desde el nodo de mayor grado como aproximación, que no necesariamente encuentra el camino más largo del grafo.

El grafo tiene 206 componentes conexas, lo que significa que hay algunas intersecciones o grupos de intersecciones que no están conectadas con la red principal, probablemente por ser caminos privados o errores en el dataset.

Algoritmos implementados.

Se implementaron tres algoritmos principales:

- **BFS (Búsqueda en Amplitud).**

BFS recorre el grafo nivel por nivel usando una cola, que sirve para encontrar el camino con menos saltos entre dos nodos sin considerar los pesos. La complejidad es $O(V+E)$ donde V son los vértices y E las aristas. En este grafo se tardó alrededor de 60ms por consulta.

- **Dijkstra.**

Dijkstra encuentra el camino más corto considerando los pesos de las aristas, usando una cola de prioridad para siempre procesar primero el nodo con menor distancia acumulada. La complejidad es $O((V+E) \log V)$ y tardó alrededor de 280ms por consulta, más lento que BFS porque tiene que manejar los pesos.

La diferencia entre Dijkstra y BFS se ve claramente en los resultados: por ejemplo en Q1, BFS encontró el camino en 101 saltos pero Dijkstra encontró uno de distancia 507, que puede tener más saltos pero menor costo total en pesos.

- **Kruskal.**

Kruskal construye el árbol generador mínimo ordenando todas las aristas por peso y agregándolas una a una siempre que no formen ciclo, todo esto para verificar si se forma ciclo usa una estructura Union-Find. La complejidad es $O(E \log E)$ por el ordenamiento.

Resultados Módulo B.

Se procesaron 10 consultas punto a punto. Los resultados fueron:

Q1: 1 -> 500000 | Dijkstra: 507 | BFS: 101 saltos | t_D: 346ms | t_B: 66ms
Q2: 100 -> 1000000 | Dijkstra: 2376 | BFS: 484 saltos | t_D: 312ms | t_B: 60ms
Q3: 50000 -> 750000 | Dijkstra: 2585 | BFS: 545 saltos | t_D: 265ms | t_B: 53ms
Q4: 200000 -> 800000 | Dijkstra: 613 | BFS: 124 saltos | t_D: 282ms | t_B: 57ms
Q5: 300000 -> 100000 | Dijkstra: 3068 | BFS: 666 saltos | t_D: 278ms | t_B: 55ms
Q6: 1 -> 1087562 | Dijkstra: 451 | BFS: 95 saltos | t_D: 282ms | t_B: 59ms
Q7: 500000 -> 1 | Dijkstra: 507 | BFS: 101 saltos | t_D: 286ms | t_B: 62ms
Q8: 250000 -> 600000 | Dijkstra: 1420 | BFS: 300 saltos | t_D: 278ms | t_B: 60ms
Q9: 10000 -> 900000 | Dijkstra: 848 | BFS: 169 saltos | t_D: 267ms | t_B: 56ms
Q10: 400000 -> 150000 | Dijkstra: 791 | BFS: 167 saltos | t_D: 270ms | t_B: 79ms

Algunas observaciones sobre los resultados:

- Dijkstra siempre tardó más que BFS, lo cual tiene sentido porque maneja más información (los pesos). En promedio Dijkstra tardó unas 5 veces más que BFS.
- Q7 es el inverso de Q1 (500000->1 vs 1->500000) y dio exactamente el mismo resultado, lo que confirma que el grafo es no dirigido.
- Q5 fue la consulta más larga con 666 saltos, lo que tiene sentido geográficamente porque conecta puntos muy alejados en la red vial.

Resultados Módulo C.

Se construyó el subgrafo inducido por los nodos de los caminos Q1 y Q6.

- Camino Q1 tiene 111 nodos
- Camino Q6 tiene 107 nodos
- El subgrafo resultante tiene 216 nodos y 222 aristas

Como el subgrafo tiene más aristas que nodos menos 1 ($222 > 215$), necesariamente tiene ciclos, lo que se confirmó con el algoritmo DFS, por lo tanto, no es un DAG. Esto tiene sentido porque los caminos Q1 y Q6 comparten algunos nodos pues ambos salen del nodo 1 y al unirlos se forman ciclos.

El peso total del MST calculado con Kruskal fue 953, que representa el costo mínimo para conectar todos los 216 nodos del subgrafo, usando solo 215 aristas ($n-1$) y sin formar ciclos.

Conclusiones.

Se puede afirmar que la diferencia entre BFS y Dijkstra es diferente en la práctica comparado con la teoría. En teoría, los dos encuentran caminos pero BFS minimiza saltos y Dijkstra minimiza peso total, en un grafo real de carreteras eso significa que BFS encuentra la ruta con menos intersecciones y Dijkstra la de menor distancia total, que no siempre son lo mismo.

Se observó que Dijkstra sobre un millón de nodos tardó solo unos 280ms gracias al heap mínimo, que sin él tardaría $O(V^2)$, lo cual sería inviable.

Se puede afirmar que la parte más difícil fue entender la reindexación de los nodos del dataset, porque los IDs originales no son consecutivos y eso causaba problemas al usar vectores.

Se puede concluir que la práctica permite entender que los algoritmos de grafos vistos en clase no son solo teoría, sino que se usan en problemas reales como navegación GPS o análisis de redes de transporte.

Referencias.

- Leskovec, J. & Krevl, A. (2014). SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- Material del curso: S08_EDNoLineales_Grafos.ipynb - Carlos Alberto Alvarez Henao, Universidad EAFIT.
- Claude (Anthropic) - apoyo en implementación del heap mínimo y estructura Union-Find.